



C++26

A Curated Tour of What's New

MARC GREGOIRE



20
26



Marc Grégoire

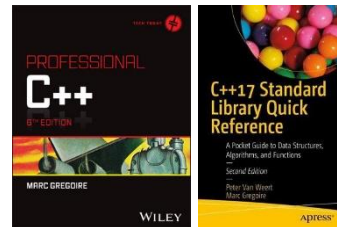
- Belgium
- Software project manager / architect at Nikon Metrology
- marc.gregoire@nuonsoft.com



- Microsoft VC++ MVP Since 2007



- Author of [Professional C++, 2nd - 6th Editions](#)
- Co-author of [C++ Standard Library Quick Reference](#) & [C++17 Standard Library Quick Reference](#)



- Founder of the [Belgian C++ Users Group](#) (BeC++)



March 2026: Technical work on C++26 finished!*



C++26 is big!
Lots of new features!

Herb Sutter: *"The most compelling release since C++11"*

** DIS Ballot voting still needs to happen.*

C++26 – Top Four Features

- Reflection
- Contracts
- Execution control library (`std::execution`)
- Less UB for more memory safety
 - ▣ C++ code is more memory safe just by recompiling as C++26

Agenda

□ C++26 Core Language

- ▣ Static Reflection
- ▣ Contracts
- ▣ Unnamed Placeholder Variables (␣)
- ▣ User-Generated static_assert() Messages
- ▣ = delete("reason") Syntax
- ▣ Attributes on Structured Bindings
- ▣ Structured Bindings in Conditions
- ▣ constexpr Structured Bindings
- ▣ Structured Bindings Introducing Packs
- ▣ Pack Indexing
- ▣ #embed
- ▣ Variadic Friend Declarations
- ▣ constexpr Casts from void*
- ▣ constexpr Placement new
- ▣ constexpr Exceptions
- ▣ Uninitialized Read Formalization

□ C++26 Standard Library

- ▣ Execution Control Library
- ▣ std::inplace_vector
- ▣ std::hive
- ▣ std::submdspan()

- ▣ Saturation Arithmetic
- ▣ string_view init for stringstreams/bitset
- ▣ std::text_encoding
- ▣ Native File Handles
- ▣ More constexpr
- ▣ New SI Prefixes
- ▣ <debugging>
- ▣ <linalg>
- ▣ Tuple Protocol for std::complex
- ▣ String Concatenation
- ▣ Enabling List Initialization for Algorithms
- ▣ views::concat
- ▣ std::ranges::cache_latest_view
- ▣ std::ranges::generate_random
- ▣ std::optional Becomes a View
- ▣ std::philox_engine
- ▣ Filesystem Path Formatter
- ▣ std::println() Blank Lines
- ▣ std::simd
- ▣ std::dynamic_format
- ▣ std::indirect
- ▣ std::polymorphic
- ▣ std::function_ref

C++26 Core Language

Static Reflection

- Initial set of features to support static reflection:
 - ▣ `std::meta::info` represents program elements via constant expressions, called reflection values
 - ▣ A reflection operator (`^^`): creates a reflection value for its operand
 - ▣ `constexpr` metafunctions that work on reflection values
 - ▣ Splicers producing grammatical elements from reflection values (`[:reflection_value:]`)

Static Reflection

□ Example:

```
constexpr auto r = ^^int;
```

```
typename[:r:] x = 42;
```

```
typename[:^^int:] c = 12;
```

```
// int x = 42;
```

```
// int c = 12;
```

Static Reflection

□ Example:

```
template <typename E> requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
    std::string label{ "<unknown>" };
    template for (constexpr auto& e : std::define_static_array(
        std::meta::enumerators_of(^E))) {
        if (value == [:e:]) { label = std::meta::identifier_of(e); }
    }
    return label;
}

enum Color { red, green, blue };

int main()
{
    std::println("{} ", enum_to_string(Color::red));
    std::println("{} ", enum_to_string(Color(42)));
}
```

Static Reflection

- Annotations can be added to your code
- Observable by reflection
- Syntax:

```
struct [[=SomeAnnotation]] Point  
{  
    int x, y;  
};
```

“[Open Content] C++26 Reflection on Windows with Visual Studio Code and WSL” – Koen Samyn

Monday, September 14 • 12:30 MDT

“Zero Cost Scripting Languages for Game Engines With C++26 Static Reflection” – Koen Samyn

Monday, September 14 • 16:45 MDT

“Can I memcpy This Type Across a Boundary? Verifying Object Representation at Compile Time With C++26 Reflection” – Fanchen Su

Tuesday, September 15 • 09:00 MDT

“Generating Language Bindings using C++ Reflection” – Callum Piper

Thursday, September 17 • 14:00 MDT

“A Duck Does More Than Quack: Synthesizing VTables with C++26 Reflection” –

Ryan Keane

Thursday, September 17 • 16:45 MDT

“What can C++26 reflection do for you?: A practical experiment in using reflection with an emphasis on binary protocol support” – Andy Webber

Friday, September 18 • 10:30 MDT

Contracts

- Three types of contract assertions
 - ▣ Precondition assertions
 - ▣ Postcondition assertions
 - ▣ Assertion statements

Contracts

□ Precondition assertions

```
int f(int i)
    pre(i > 0);
```

```
int f(int i)
    pre(i > 0 && "i must be greater than 0");
```

Contracts

- Postcondition assertions

```
void MyContainer::clear()  
    post(empty());
```

- Can specify a result binding that names the result object of the function

```
int f(const int i)  
    post(r: r >= i); // r refers to the result object of f.
```

Contracts

- Pre and postconditions can be combined

```
void f()  
    pre(a)  
    post(b)  
    pre(c);
```

Contracts

- Assertion statements

```
contract_assert(x != 3);
```

Contracts

```
int f(const int x)
    pre(x != 1)                // A precondition assertion
    post(r : r == x && r != 2) // A postcondition assertion
                                // r names the result object of f
{
    contract_assert(x != 3);    // An assertion statement
    return x;
}
```

```
f(0); // No violations
f(1); // Violates precondition assertion of f
f(2); // Violates postcondition assertion of f
f(3); // Violates assertion statement within f
```

Contracts

- Four evaluation semantics:
 - ▣ **Ignore**: assertions are ignored.
 - ▣ **Observe**: Contract violation →
 - invoke the contract-violation handler
 - if it returns normally, program execution is continued
 - ▣ **Enforce**: Contract violation →
 - invoke the contract-violation handler
 - if it returns normally, program is terminated
 - ▣ **Quick-enforce**: Contract violation →
 - **do not** invoke the contract-violation handler
 - Program is contract-terminated

Contracts

- ❑ The contract-violation handler is called `::handle_contract_violation`
- ❑ It accepts an argument of type `std::contracts::contract_violation` with details of the violation
- ❑ The standard recommends implementations to at least output a diagnostics message
- ❑ It's implementation-defined if a user can replace this handler

**“Towards a complete contract-assertion
facility for C++” – Timur Doumler**
Monday, September 14 • 14:00 MDT

Unnamed Placeholder Variables (_)

- Use _ as placeholder for unnamed entities

- Instead of:

```
std::lock_guard namingIsHard(mutex);
```

```
auto [x, y, iDontCare] = f();
```

- Write:

```
std::lock_guard _(mutex);
```

```
auto [x, y, _] = f();
```

User-Generated `static_assert()` Messages

- C++26, 2nd argument can be any constant-evaluated string
 - ▣ e.g.: `std::string_view` from a `constexpr` function
- Before C++26, 2nd argument of `static_assert()` had to be a string literal:

```
static_assert(true, "Error message");
```

User-Generated static_assert() Messages

- Allows for sharing of strings

```
constexpr std::string_view msg { "Error message" };  
static_assert(true, msg);  
static_assert(..., msg);
```

- Allows for computed strings

Future example

```
static_assert(sizeof(S) == 1,  
    std::format("Unexpected sizeof: expected 1, got {}",  
        sizeof(S)));
```

This will work once `std::format()` is `constexpr`

= delete("reason") Syntax

- Provide custom message for deleted functions
- Example

```
void newapi();
```

```
void oldapi() = delete("Old API is removed. Use newapi().");
```

= delete("reason") Syntax

□ Example

```
struct MoveOnly
{
    // Move members
    // ...
    // Copy members
    MoveOnly(const MoveOnly&) = delete("Copy-construction is
        expensive; use move construction instead.");

    MoveOnly& operator=(const MoveOnly&) = delete("Copy-assignment is
        expensive; use move assignment instead.");
};
```

Attributes on Structured Bindings

- Apply attributes to individual introduced structured bindings
- Example:

```
auto [x, [[maybe_unused]] y] = std::pair{ 1, 2 };
```

Structured Bindings in Conditions

- ❑ Before C++26, structured bindings can be used as a statement on its own or in range-based **for** loops
- ❑ C++26 adds support for structured bindings as the condition in **if**, **while**, **for**, and **switch** statements
- ❑ Example:

```
if (auto [a, b, c] = f()) {}
```
- ❑ Roughly equivalent to:

```
{ auto e = f();  
if (auto [a, b, c] = e; static_cast<bool>(e)) {} }
```

Structured Bindings in Conditions

- Example using `std::to_chars()`

- Instead of:

```
if (auto [ptr, ec] = std::to_chars(p, last, 4); ec == std::errc{}) {  
    // success  
} else {  
    // error  
}
```

- You can write:

```
if (auto [ptr, ec] = std::to_chars(p, last, 4)) {  
    // success  
} else {  
    // error  
}
```

- Uses the new C++26 operator `bool` for `std::to_chars_result`

constexpr Structured Bindings

- Structured bindings can now be declared **constexpr**

- Example:

```
constexpr auto [a] = std::tuple(1);
```

Structured Bindings Introducing Packs

- Allows introducing a pack using a structured binding (only one pack can be introduced)

- Examples:

```
std::tuple<X, Y, Z> f();
```

```
auto [x, y, z] = f();
```

```
auto [...p] = f();
```

```
auto [x, ...rest] = f();
```

```
auto [x, y, z, ...rest] = f();
```

```
auto [...a, ...b] = f();
```

```
// OK today
```

```
// p is a pack of length 3
```

```
// containing an X, Y, and a Z
```

```
// x is an X, rest is a pack of
```

```
// length 2 (Y and Z)
```

```
// rest is an empty pack
```

```
// ill-formed: multiple packs
```

```
// not allowed
```

Pack Indexing

- Allows accessing specific elements in parameter packs

```
void bar(auto... values) {  
    auto a = values...[0];  
  
    const int n = 1;  
    auto b = values...[n];  
  
    int m = 2;  
    auto c = values...[m]; // error: 'm' not a constant expression  
}
```

Pack Indexing

```
template <std::size_t I, typename... Ts>  
constexpr auto element_at(Ts... args)  
{  
    return args...[I];  
}
```

```
static_assert(element_at<0>(3, 5, 9) == 3);  
static_assert(element_at<2>(3, 5, 9) == 9);  
static_assert(element_at<3>(3, 5, 9) == 4); // out of bounds
```

Pack Indexing

```
template <std::size_t I, typename Tup>
constexpr auto structured_binding_element_at(Tup tup)
{
    auto [...elems] = tup;
    // 'elems' introduced in structured binding pack declaration
    return elems...[I];
}
```

```
struct A { bool a; int b; };
```

```
static_assert(structured_binding_element_at<0>(A{ true, 4 }) == true);
static_assert(structured_binding_element_at<1>(A{ true, 4 }) == 4);
```

#embed

- Allows for easy pulling in binary data

- Example:

```
const unsigned char icon_display_data[] = {  
    #embed "art.png"  
};
```

- For infinite resource, you can specify a limit:

```
const int random_data[] = {  
    #embed "/dev/urandom" limit(512)  
};
```

Variadic Friend Declarations

- Allows granting friendship to all classes in a parameter pack

- Instead something like:

```
template<class T = void, class U = void>
class Foo {
    friend T;
    friend U;
};
```

- You can now write:

```
template<class... Ts>
class Foo {
    friend Ts...;
};
```

constexpr Casts from void*

- ❑ Cast from **void*** to **T*** in **constexpr** contexts
 - ❑ Only if the type of the object at the **void*** address is of type **T**
- ❑ Conversions to interconvertible (including pointers to base classes) or unrelated types are not allowed
- ❑ Allows **constexpr** type erasure

constexpr Casts from void*

```
struct Sheep { constexpr std::string_view speak() const noexcept { return "Baaaaaa"; } };
struct Cow { constexpr std::string_view speak() const noexcept { return "Mooo"; } };

class Animal_View {
public:
    template <typename Animal> constexpr Animal_View(const Animal& a) : animal{ &a },
        speak_function{ [] (const void* object) { return static_cast<const Animal*>(object)->speak(); } }
    {
    }
    constexpr std::string_view speak() const noexcept { return speak_function(animal); }
private:
    const void* animal;
    std::string_view (*speak_function)(const void*);
};

// This is the key bit here. This is a single concrete function
// that can take anything that happens to have the "Animal_View"
// interface
constexpr std::string_view do_speak(Animal_View av) { return av.speak(); }

int main() {
    constexpr Cow cow;
    constexpr auto result = do_speak(cow); // <-- Pre-C++26: cannot be constexpr because of static_cast
    return static_cast<int>(result.size());
}
```

constexpr Casts from void*

- Allows to make several features **constexpr** in the future:
 - ▣ `std::format()`
 - ▣ `std::function_ref`
 - ▣ `std::function`
 - ▣ `std::any`

constexpr Placement new

- Placement **new** expression in constant evaluation
- Example of placement new syntax:

```
int buf[1] = {};  
int* p = new (buf) int(42);
```

constexpr Exceptions

- Enable throw/catch in constant evaluation

```
constexpr double divide(double n, double d) {  
    if (d == 0.0) { throw std::invalid_argument{ "division by zero" }; }  
    return n / d;  
}
```

```
constexpr auto b = divide(5., 0.); // Before C++26: compilation error due to  
                                   // reaching a throw expression  
                                   // C++26: still a compilation error but  
                                   // due to the uncaught exception
```

```
constexpr std::optional<double> checked_divide(double n, double d) {  
    try { return divide(n, d); } catch (...) { return std::nullopt; }  
}
```

```
constexpr auto a = checked_divide(5., 0.); // Before C++26: compilation error  
                                           // C++26: std::nullopt value
```

Uninitialized Read Formalization

- ❑ C++26 introduces a new behavior:
- ❑ **Erroneous behavior**
 - ▣ Erroneous behavior is always the consequence of incorrect program code
 - ▣ Implementations are allowed, but not required, to diagnose it.
- ❑ Many evaluations that read an erroneous value are erroneous behavior.
 - ▣ C++26 formalizes these cases and distinguishes them from indeterminate values and undefined behavior.
- ❑ Example

```
int d1;  
int e1 = d1; // Erroneous behavior
```

Uninitialized Read Formalization

- New attribute `[[indeterminate]]`
- Specifies that storage of an object with automatic storage duration is initially indeterminate rather than erroneous

- Example:

```
void f(int);
```

```
void g() {  
    int y;  
    f(y);      // erroneous behavior, reading uninitialized variable  
  
    int x [[indeterminate]];  
    f(x);      // undefined behavior  
}
```

Uninitialized Read Formalization

□ Example:

```
int x [[indeterminate]];
std::cin >> x;
```

Agenda

□ C++26 Core Language

- ▣ Static Reflection
- ▣ Contracts
- ▣ Unnamed Placeholder Variables ()
- ▣ User-Generated static_assert() Messages
- ▣ = delete("reason") Syntax
- ▣ Attributes on Structured Bindings
- ▣ Structured Bindings in Conditions
- ▣ constexpr Structured Bindings
- ▣ Structured Bindings Introducing Packs
- ▣ Pack Indexing
- ▣ #embed
- ▣ Variadic Friend Declarations
- ▣ constexpr Casts from void*
- ▣ constexpr Placement new
- ▣ constexpr Exceptions
- ▣ Uninitialized Read Formalization

□ C++26 Standard Library

- ▣ Execution Control Library
- ▣ std::inplace_vector
- ▣ std::hive
- ▣ std::submdspan()

- ▣ Saturation Arithmetic
- ▣ string_view init for stringstreams/bitset
- ▣ std::text_encoding
- ▣ Native File Handles
- ▣ More constexpr
- ▣ New SI Prefixes
- ▣ <debugging>
- ▣ <linalg>
- ▣ Tuple Protocol for std::complex
- ▣ String Concatenation
- ▣ Enabling List Initialization for Algorithms
- ▣ views::concat
- ▣ std::ranges::cache_latest_view
- ▣ std::ranges::generate_random
- ▣ std::optional Becomes a View
- ▣ std::philox_engine
- ▣ Filesystem Path Formatter
- ▣ std::println() Blank Lines
- ▣ std::simd
- ▣ std::dynamic_format
- ▣ std::indirect
- ▣ std::polymorphic
- ▣ std::function_ref

C++26 Standard Library

Execution Control Library

- ❑ A new library to manage asynchronous execution on generic execution resources.
- ❑ Defined in `<execution>`
- ❑ Asynchrony is used everywhere: networking, file I/O, GUIs, ...
- ❑ Existing **`std::async`**, **`future`**, **`promise`** is not always easy to use.

Execution Control Library

- Key design choices:
 - ▣ Building blocks must be composable and generic to allow code to work with different types of execution resources.
 - ▣ Common asynchronous patterns must be encapsulated in reusable algorithms.
 - ▣ It must be easy to be used correctly by construction.
 - ▣ Care must taken about all reasonable use cases, domains, platforms.
 - ▣ Errors must be propagated correctly; error handling should not be a burden.
 - ▣ Cancellation must be supported.

Execution Control Library

- A few key abstractions:
 - ▣ **Execution resources**: describe a place where work is executed (thread pool, GPU, ...)
 - ▣ **Schedulers**: a lightweight handle representing a strategy for scheduling work on some execution resource.
 - ▣ **Senders**: describe asynchronous work and send a signal (a value, an error, or stopped) to a connected receiver when the asynchronous work finishes.
 - ▣ **Receivers**: serve as the glue between senders
 - A callback supporting `set_value()`, `set_error()`, and `set_stopped()`.
 - Similar to `std::promise` which supports `set_value()` and `set_exception()`.
 - ▣ Scheduler / sender are defined in the standard as concepts

Execution Control Library

□ Asynchronous algorithms

- ▣ **Sender factories:** algorithms taking no senders as parameters and returning a sender.

Examples:

- **execution::schedule:** a sender describing the start of a task graph.
- **execution::just:** a sender without completion schedulers, just sending a given value.
- **execution::just_error:** a sender without completion schedulers, just sending a given error.
- ...

Execution Control Library

□ Asynchronous algorithms

- **Sender adaptors**: algorithms taking one or more senders as parameters and returning a sender whose completion is related to the given senders.

Examples:

- **execution::continues_on**: a sender to transition from one execution agent to another one.
- **execution::then**: a sender invoking a given callable with values send by an input sender.
- Many more: **execution::starts_on**, **execution::when_all**, ...

Execution Control Library

- Asynchronous algorithms
 - ▣ **Sender consumers:** algorithms taking one or more senders as parameters and not returning a sender.
Examples:
 - **this_thread::sync_wait:** a sender consumer submitting the work provided by an input sender for execution and blocking for its completion.

Execution Control Library

□ Example:

```
using namespace std::execution;

scheduler auto pool_scheduler = thread_pool.scheduler();    // 1

std::execution::sender auto snd =
    std::execution::schedule(pool_scheduler)                // 2
    | std::execution::then([] {                             // 3
        std::println("Hello world! Have an int.");         // 3
        return 13;                                         // 3
    })
    | std::execution::then([](int arg) { return arg + 42; }); // 4

auto [i] = std::this_thread::sync_wait(snd).value();       // 5
```

std::inplace_vector

- ❑ Defined in `<inplace_vector>`
- ❑ A dynamically-resizable vector
- ❑ Capacity fixed at compile time
- ❑ Contiguous inplace storage
 - ▣ Elements are stored within the vector object itself
- ❑ Useful when:
 - ▣ Memory allocation is not possible (e.g. embedded systems)
 - ▣ Memory allocation is expensive
 - ▣ Storage of elements must be inplace, e.g. for serialization with `memcpy()`

std::inplace_vector

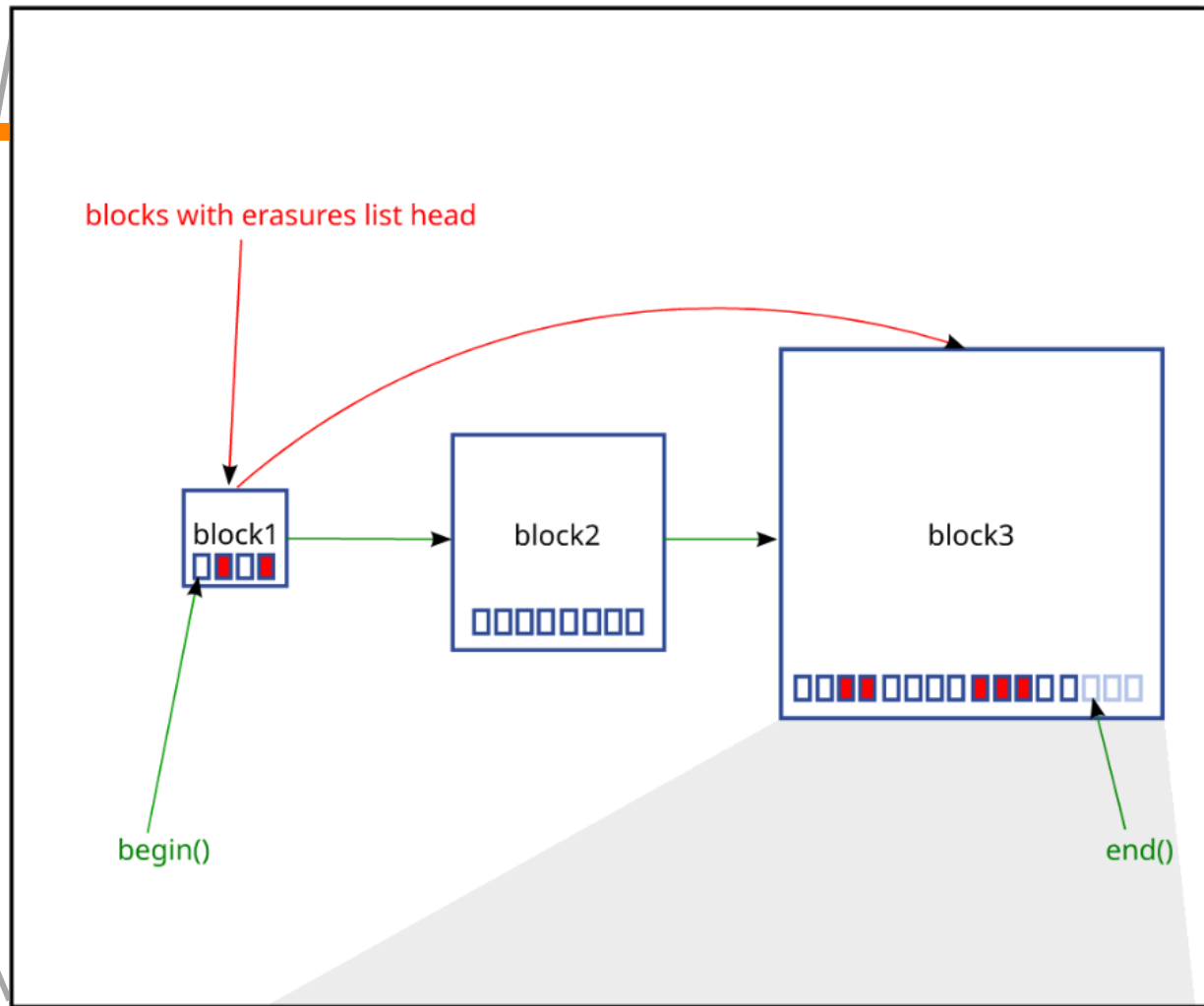
- Difference with `std::array`
 - ▣ `std::inplace_vector<T, N>` can store up to `N` elements while `std::array<T, N>` always stores `N` elements
- Example:

```
std::inplace_vector<int, 3> v { 0, 1 }; // Capacity is 3, size is 2
v.push_back(2);           // Size is now 3
v.push_back(3);           // Throws bad_alloc, as capacity is exceeded
auto result = v.try_push_back(4);
// Returns empty optional<reference>, as capacity is exceeded
```

std::hive

- ❑ Defined in `<hive>`
- ❑ Typically known as a *bucket array* or *object pool*
- ❑ Concept:
 - ▣ Multiple memory blocks of elements.
 - ▣ A boolean token for each element which denotes whether the element is *active* or *erased*.
 - Commonly called *skipfield*: If it is *erased*, it is skipped over during iteration.
 - ▣ When all elements in a block are erased, the block is removed.
 - ▣ If an insertion occurs when all blocks are full, a new memory block is allocated.

hive



std::hive

- Benefits:
 - ▣ Use of skipfields → no reallocation of elements is necessary upon erasure.
 - ▣ Use multiple memory blocks → insertions in a full container do not trigger reallocations.
 - ▣ In other words:
 - element memory locations stay stable
 - iterators stay valid regardless of erasure/insertion

std::hive

□ Example:

```
std::hive<int> h { 1, 2 };
```

```
h.insert(11);
```

```
h.insert(22);
```

```
for (int num : h) {  
    std::print("{} ", num);  
}
```

std::submdspan()

- A slicing function returning a view (!) of a subset of an `std::mdspan`.

std::submdspan()

□ Example:

// Set all elements of a rank-2 mdspan to zero.

```
void zero_2D_grid(auto& grid2d) {  
    static_assert(grid2d.rank() == 2);  
    for (int i = 0; i < grid2d.extent(0); ++i) {  
        for (int j = 0; j < grid2d.extent(1); ++j) {  
            grid2d[i, j] = 0;  
        }  
    }  
}
```

```
void zero_3D_cube_surface(auto& grid3d) {  
    static_assert(grid3d.rank() == 3);  
    zero_2D_grid(submdspan(grid3d, 0, full_extent, full_extent));  
    zero_2D_grid(submdspan(grid3d, full_extent, 0, full_extent));  
    zero_2D_grid(submdspan(grid3d, full_extent, full_extent, 0));  
    zero_2D_grid(submdspan(grid3d, grid3d.extent(0) - 1, full_extent, full_extent));  
    zero_2D_grid(submdspan(grid3d, full_extent, grid3d.extent(1) - 1, full_extent));  
    zero_2D_grid(submdspan(grid3d, full_extent, full_extent, grid3d.extent(2) - 1));  
}
```

Saturation Arithmetic

- ❑ Unsigned integer arithmetic in C++ is performed module 2^N .
- ❑ Saturating arithmetic instead clamps results outside the representable range to the largest or smallest representable value.
- ❑ New freestanding functions in **<numeric>**:
 - ❑ `add_sat(x, y)`
 - ❑ `sub_sat(x, y)`
 - ❑ `mul_sat(x, y)`
 - ❑ `div_sat(x, y)`

Saturation Arithmetic

- Example:

Brightening a white (=255) pixel value

```
unsigned char x1 = 255 + 4; // = 3
```

```
unsigned char x2 = std::add_sat<unsigned char>(255, 4); // = 255
```

string_view init for stringstream/bitset

- Allows construction and reinitialization of `std::stringstream` and `std::bitset` from `std::string_view`
- Example:

```
std::stringstream s1{ ""s }; // OK pre-C++26
std::stringstream s2{ ""sv }; // OK since C++26
std::stringstream s3;
s3.str(""); // OK pre-C++26
s3.str("sv"); // OK since C++26
std::bitset b1{ ""s }; // OK pre-C++26
std::bitset b2{ ""sv }; // OK since C++26
```

std::text_encoding

- Introduces a way to retrieve

- ▣ Literal text encoding
- ▣ Environment text encoding

- Example:

```
bool is_UTF8_environment()  
{  
    return std::text_encoding::environment_is<std::text_encoding::id::UTF8>();  
}
```

std::text_encoding

□ Example:

```
void print_suffix(auto& stream, const auto& duration) {  
    if constexpr (std::text_encoding::literal() == std::text_encoding::id::UTF8) {  
        if (stream.getloc().encoding() == std::text_encoding::id::UTF8) {  
            stream << duration.count() << "\u00B5s"; // μ  
            return;  
        }  
    }  
    stream << duration.count() << "us";  
}  
  
int main() {  
    print_suffix(std::cout, std::chrono::microseconds(42));  
}
```

Native File Handles

- ❑ Expose OS handles in file streams
- ❑ New type alias **native_handle_type**, mapped to OS-specific type:
 - ❑ `int` on POSIX
 - ❑ `HANDLE (void*)` on Windows
 - ❑ ...
- ❑ New member `native_handle()` in `basic_filebuf`, `basic_ifstream`, `basic_ofstream`, `basic_fstream`

Native File Handles

- Example POSIX implementation using `fstat`:

```
std::chrono::sys_seconds last_modified(int fd) {  
    struct stat s {};  
    if (::fstat(fd, &s) != 0) {  
        throw std::system_error(errno, std::generic_category());  
    }  
    return std::chrono::sys_seconds{  
        std::chrono::seconds{s.st_mtime}  
    };  
}
```

```
int main() {  
    std::ofstream of("/tmp/foo");  
    auto lm = last_modified(of.native_handle());  
    std::println("{:%c}", lm);  
}
```

constexpr Stable Sorting

- The following algorithms are now **constexpr**
 - ▣ `std::stable_sort()`
 - ▣ `std::stable_partition()`
 - ▣ `std::inplace_merge()`

More constexpr <cmath> and <complex>

- More **constexpr** math function in <cmath> and <complex>
- Functions such as:
 - ▣ <complex>: `abs()`, `sin()`, `cos()`, `asin()`, `acosh()`, `log()`, `pow()`, ...
 - ▣ <cmath>: `sinhf()`, `tanf()`, `cosf()`, `expf()`, `logf()`, `powf()`, `lgammaf()`, `hypot()`, ...

More constexpr

- ❑ Most methods of `std::atomic` and `std::atomic_ref` are now `constexpr`
- ❑ `constexpr` for specialized memory algorithms like `std::uninitialized_value_construct()`, `uninitialized_copy()`, ...
- ❑ `constexpr` for standard exception types like `std::bad_alloc`, `bad_cast`, `invalid_argument` ...

constexpr for Containers & Adapters

- ❑ C++26 adds broad **constexpr** support to the standard containers and container adapters:
 - ❑ `std::deque`, `flat_map`, `list`, `set`, `priority_queue`, `stack`, `map`, `span`, `inplace_vector`, ...
- ❑ Except `std::hive`

New SI Prefixes

- ❑ Support for new SI prefixes
 - ❑ `std::quetta` (Q): 10^{30}
 - ❑ `std::ronna` (R): 10^{27}
 - ❑ `std::ronto` (r): 10^{-27}
 - ❑ `std::quecto` (q): 10^{-30}
- ❑ Only declared if `std::intmax_t` can represent the denominator or numerator (same requirement as `std::yocto`, `zepto`, `zetta`, and `yotta`)

<debugging>

- New functions
 - ▣ `std::is_debugger_present()`
 - ▣ `std::breakpoint()`
 - ▣ `std::breakpoint_if_debugging()`
 - Equivalent to:
`if (std::is_debugger_present()) { std::breakpoint(); }`

<linalg>

- ❑ Introduces a free function linear algebra interface based on BLAS (Basic Linear Algebra Subroutines).
- ❑ Only a subset of the BLAS standard.
- ❑ Supported functionality
 - ▣ Elementwise vector sums.
 - ▣ Multiplying all elements of a vector or matrix by a scalar.
 - ▣ 2-norms and 1-norms of vectors.
 - ▣ Vector-vector, matrix-vector, matrix-matrix products.
 - ▣ Low-rank updates of a matrix.
 - ▣ Triangular solves with one or more “right-hand side” vectors.
 - ▣ Generating and applying plane (Givens) rotations.

<linalg>

- Features:
 - ▣ Uses free functions, not operator overloading
 - ▣ Uses `std::mdspan` to represent matrices and vectors
 - ▣ Permits optimizations for matrices and vectors with small compile-time dimensions
 - ▣ Supports mixed-precision computations with matrices and vectors that have different element types
 - ▣ Several operations take an optional execution policy to support parallel execution
 - ▣ ...

<linalg>

□ Example:

```
constexpr size_t N = 40;
std::vector<double> x_vec(N);
std::iota(begin(x_vec), end(x_vec), 0.0);
std::mdspan x(x_vec.data(), N);           // 0, 1, 2, 3, ...
std::linalg::scale(2.0, x);               // 0, 2, 4, 6, ...
std::linalg::scale(
    std::execution::par_unseq, 3.0, x);   // 0, 6, 12, 18, ...
```

Tuple Protocol for `std::complex`

- Access `std::complex` values as tuple-like values, enabling structured bindings
- Example:

```
std::complex<double> c{1, 2};  
auto& [r, i] {c};
```
- Before C++26, `std::complex` did not participate in the tuple protocol, so structured bindings were not directly supported.

String Concatenation

- New overloads of `operator+` to allow concatenation between `std::string` and `std::string_view`

- Example:

```
std::string str = " Hello";  
std::string_view view = " World";  
std::println("{} ", str + view);  
std::println("{} ", view + str);
```

Enabling List Initialization for Algorithms

- C++26 supports list initialization for algorithms.

- Example:

```
struct point {int x; int y; bool operator==(const point&) const = default;};  
std::vector<point> v;  
v.push_back({ 3, 4 }); // No need to specify the "point" type here...
```

- Instead of:

```
auto res1 { std::find(v.begin(), v.end(), point{ 3, 4 }) };  
std::ranges::fill(v, point{ 3, 4 });
```

- You can now write:

```
auto res2 { std::find(v.begin(), v.end(), { 3, 4 }) };  
std::ranges::fill(v, { 3, 4 });
```

views::concat

- ❑ `std::views::concat` is a new view factory
 - ▣ Takes any number of input ranges
 - ▣ Returns a view containing all elements of all ranges concatenated
- ❑ Example:

```
std::vector<int> v1{ 1, 2, 3 }, v2{ 4, 5 }, v3{};
std::array a{ 6, 7, 8 };
std::println("{} ", std::views::concat(v1, v2, v3, a));
// [1, 2, 3, 4, 5, 6, 7, 8]
```

std::ranges::cache_latest_view

- View adapter caching the result of the last dereference of the underlying iterator
- Example:

```
auto square = [](int i) { std::print("transform: {} ", i); return i * i; };  
auto is_even = [](int i) { std::print("filter: {} ", i); return i % 2 == 0; };  
std::vector v = { 1, 2, 3, 4, 5 };  
auto even_squares = v  
    | std::views::transform(square)  
    | std::views::filter(is_even);  
for (int i : even_squares) { std::println("Got: {}", i); }
```

```
transform: 1 filter: 1 transform: 2 filter: 4 transform: 2 Got: 4  
transform: 3 filter: 9 transform: 4 filter: 16 transform: 4 Got: 16  
transform: 5 filter: 25
```

std::ranges::cache_latest_view

```
auto square = [](int i) { std::print("transform: {} ", i); return i * i; };
auto is_even = [](int i) { std::print("filter: {} ", i); return i % 2 == 0; };
std::vector v = { 1, 2, 3, 4, 5 };
auto even_squares = v
    | std::views::transform(square)
    | std::views::cache_latest
    | std::views::filter(is_even);
for (int i : even_squares) { std::println("Got: {}", i); }
```

```
transform: 1 filter: 1 transform: 2 filter: 4 Got: 4
transform: 3 filter: 9 transform: 4 filter: 16 Got: 16
transform: 5 filter: 25
```

std::ranges::generate_random

- New algorithm to generate random numbers

- Example:

```
std::array<int, 42> random_ints;  
std::mt19937 gen { std::random_device{}()};  
std::uniform_int_distribution<int> dist { 0, 100 };  
std::ranges::generate_random(random_ints, gen, dist);
```

std::optional Becomes a View

- std::optional is now a view
- Empty optional produces zero elements
- Example:

```
std::optional<int> value = 42;
for (int x : value) {
    std::println("{} ", x);
}
auto result = value |
    std::views::transform([](int x) { return x * 2; });
std::println("{} ", result); // [84]
```

std::philox_engine

- ❑ C++26 adds counter-based Philox engines to `<random>`
 - ❑ `philox_engine`
 - ❑ `philox4x32` and `philox4x64` predefined engines
- ❑ Benefits:
 - ❑ Small state (e.g. **Philox4x32** has ten 32-bit elements)
 - ❑ Long period (e.g. **Philox4x32** has period 2^{130})
 - ❑ Easy to vectorize and parallelize
 - ❑ Well suited for CPU/GPU/FPGA/...
 - ❑ Already broadly used in Monte-Carlo simulations

std::philox_engine

□ Example:

```
std::philox4x32 gen { std::random_device{}() };  
std::normal_distribution nd;  
std::println("{} ", nd(gen));
```

Filesystem Path Formatter

- C++26 supports formatting of `std::filesystem::path`

- Example:

```
auto p { std::filesystem::path("/usr/bin") };  
std::cout << std::format("{} ", p); // /usr/bin
```

- Example on Windows with UTF-8 as a literal encoding:

```
auto p { std::filesystem::path(L"Щучыншчына") };  
std::print("{} ", p); // Щучыншчына
```

std::println() Blank Lines

- Easily print blank lines

- Instead of:

```
std::println("");
```

- Write:

```
std::println();
```

std::simd

- ❑ Defined in `<simd>`
- ❑ Defines data-parallel types and operations on those types in a portable manner
- ❑ Types `vec`, `mask`, ...
- ❑ Operations such as `simd::min`, `clamp`, `reduce`, `all_of`, ...
- ❑ Overloaded math function such as `pow`, `cos`, `sin`, ...
- ❑ Take advantage of SIMD instructions of CPUs

std::simd

□ Example:

```
constexpr std::simd::vec<double, 16> x([](int i) {return i;});  
for (auto v : x) { std::print("{} ", v); }  
// x: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15  
  
// Overloaded math functions defined in <simd>  
std::println("cos2(x) + sin2(x): {}",  
    std::pow(std::cos(x), 2) + std::pow(std::sin(x), 2));  
// cos2(x) + sin2(x): 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

**“std::simd Without Compromise:
Making SIMD in C++26 and Beyond
as Fast as Silicon Allows” –**

Ruslan Arutyunyan & Daniel Towner
Monday, September 14 • 15:15 MDT

std::dynamic_format

- Allows for runtime formatting strings

- Example:

```
std::string format_message(std::string_view fmt, std::string_view name)
{
    return std::format(std::dynamic_format(fmt), name);
}
```

```
int main()
{
    std::string fmt = "Hello, {}!";
    std::cout << format_message(fmt, "CppCon");
}
```

std::indirect

- Solves the following use case:
 - ▣ Allocate non-polymorphic types on the heap
 - ▣ Easily make copies

- Example:

```
struct Coordinate { int x{}; int y{}; int z{}; };  
std::indirect<Coordinate> col{ std::in_place, 1, 0, 0 };  
std::println("{} ", col->x); // Use as a pointer // Output: 1
```

```
auto co2 = col; // Easy to make a copy  
col->x = 100;  
std::println("{} {}", col->x, co2->x); // Output: 1 100
```

std::polymorphic

- Solves the following use case:
 - ▣ Allocate polymorphic types on the heap
 - ▣ Easily make copies

- Example:

```
class Base { /*...*/ };  
class Derived1 : public Base { /*...*/ };  
class Derived2 : public Base { /*...*/ };
```

```
std::vector<std::polymorphic<Base>> storedItems;  
storedItems.emplace_back(Derived1{});  
storedItems.emplace_back(Derived2{});
```

```
auto copies = storedItems;
```

std::function_ref

- ❑ `std::function` owns a callable
- ❑ `std::function_ref` is a lightweight, non-owning callable reference, trivially copyable
- ❑ `std::function_ref` is to `std::function` roughly what `std::span` is to `std::vector`: a lightweight, non-owning view

Callable-Wrappers

- C++26 now supports the following callable-wrappers

Type	Owns callable?	Copyable?	Typical use
<code>std::function</code>	Yes	Yes	General-purpose owning type-erased callable
<code>std::move_only_function</code>	Yes	No	Owning move-only callable
<code>std::copyable_function</code>	Yes	Yes	Owning callable with improved cv/ref/noexcept signature support
<code>std::function_ref</code>	No	Yes	Lightweight non-owning callable reference

Agenda

□ C++26 Core Language

- ▣ Static Reflection
- ▣ Contracts
- ▣ Unnamed Placeholder Variables ()
- ▣ User-Generated static_assert() Messages
- ▣ = delete("reason") Syntax
- ▣ Attributes on Structured Bindings
- ▣ Structured Bindings in Conditions
- ▣ constexpr Structured Bindings
- ▣ Structured Bindings Introducing Packs
- ▣ Pack Indexing
- ▣ #embed
- ▣ Variadic Friend Declarations
- ▣ constexpr Casts from void*
- ▣ constexpr Placement new
- ▣ constexpr Exceptions
- ▣ Uninitialized Read Formalization

□ C++26 Standard Library

- ▣ Execution Control Library
- ▣ std::inplace_vector
- ▣ std::hive
- ▣ std::submdspan()

- ▣ Saturation Arithmetic
- ▣ string_view initialization
- ▣ std::text_encoding
- ▣ Native File Handling
- ▣ More constexpr
- ▣ New SI Prefixes
- ▣ <debugging>
- ▣ <inalg>
- ▣ Tuple Protocol for std::complex
- ▣ String Concatenation
- ▣ Enabling List Initialization
- ▣ views::concat
- ▣ std::ranges::cache
- ▣ std::ranges::generate_random
- ▣ std::optional Backends
- ▣ std::philox_engine
- ▣ Filesystem Path
- ▣ std::println() Blank
- ▣ std::simd
- ▣ std::dynamic_for
- ▣ std::indirect
- ▣ std::polymorphic
- ▣ std::function_ref

“[Open Content] One C++26 App Three Ways: By Hand, By AI, and Together” –
Mike Spertus
Tuesday, September 15 • 18:30 MDT

“Breaking the Speed Limit: Building a High-Throughput Hashing Engine With C++26” – Yannic Staudt
Wednesday, September 16 • 09:00 MDT

“How C++26 changes the way we write code” – Timur Doumler
Wednesday, September 16 • 10:30 MDT

“The Address is Not The Place: Object Residency in C++26” – Laurie Kirk
Thursday, September 17 • 10:30 MDT

“The Real Story of C++26: Beyond the Headline Features” – Sandor Dargo
Friday, September 18 • 13:30 MDT

Even More Features/Changes

□ C++26 Core Language

- Unevaluated strings
- Adding @, \$, and ` to the basic character set
- Removing deprecated arithmetic conversion on enumerations
- Disallow binding a returned reference to a temporary
- Ordering of constraints involving fold expressions
- Deleting a pointer to an incomplete type should be ill-formed
- Oxford variadic comma
- Removing deprecated array comparisons
- Concept and variable-template template-parameters
- Trivial relocatability
- Trivial unions
- Observable checkpoints
- ...

Even More Features/Changes

□ C++26 Standard Library

- ▣ Arithmetic overloads of `std::to_string` and `std::to_wstring` use `std::format`
- ▣ Type checking format args
- ▣ Formatting pointers
- ▣ Hashing support for `std::chrono` value classes
- ▣ `<rcu>`: Read-Copy Update
- ▣ `<hazard_pointer>`: Hazard pointers
- ▣ ADL-proof `std::projected`
- ▣ Member `std::basic_format_arg::visit()`
- ▣ Member `std::variant::visit()`
- ▣ `std::is_within_lifetime`
- ▣ Extending associative containers with the remaining heterogeneous overloads
- ▣ Enabling the use of `std::weak_ptr` as keys in unordered associative containers
- ▣ `std::bind_front`, `std::bind_back`, and `std::not_fn` to NTTP callables
- ▣ Freestanding language: Optional `::operator new`
- ▣ Freestanding `std::char_traits`, `std::errc` and facilities in: `<charconv>`, `<cstdlib>`, `<cstring>`, `<wchar>`
- ▣ Making `assert()` macro user friendly
- ▣ `std::span` over an initializer list
- ▣ `std::span::at()`
- ▣ A new specification for `std::generate_canonical`
- ▣ More freestanding changes
- ▣ Comparisons for `std::reference_wrapper`, and constraints for comparisons of `std::pair`, `std::tuple`, `std::optional`, and `std::variant`
- ▣ Padded `mdspan` layouts
- ▣ New CTAD for `std::span` and `std::mdspan` with integral constants
- ▣ Atomic maximum and minimum
- ▣ Undeprecate `std::polymorphic_allocator::destroy`
- ▣ `std::is_virtual_base_of`: type trait for detecting virtual base classes
- ▣ `std::dims` for `std::mdspan`
- ▣ Deprecating `std::is_trivial` and `std::is_trivial_v`
- ▣ Constraining `std::expected` equality operators
- ▣ `std::aligned_accessor`: an `std::mdspan` accessor expressing pointer over-alignment
- ▣ `std::is_sufficiently_aligned`: checks a pointer's alignment precondition
- ▣ `std::atomic_ref::address()`
- ▣ `<stdbit.h>` and `<stdckdint.h>`
- ▣ Trivial relocatability traits and operations
- ▣ Defang and deprecate `std::memory_order::consume`
- ▣ `std::ranges::to_input_view`
- ▣ `std::ranges::approximately_sized_range` and `std::ranges::reserve_hint`
- ▣ Standard library hardening
- ▣ ...

Questions

